



General Sir John Kotelawala Defense University, Sri Lanka

Faculty Of Engineering

ET3112 Assignment 02 on Fitting

Student Name : WALP Harsha

Registration Number : D/ENG/24/0095/ET

Instructed By : Dr. Sampath Perera

Stream/Intake : Intake 41

Module Code/Name : ET3112-Image Processing and Machine Vision

Date Of Submission : 20.03.2026

GitHub repo Link : [Assignment2 NeW Repository](#)

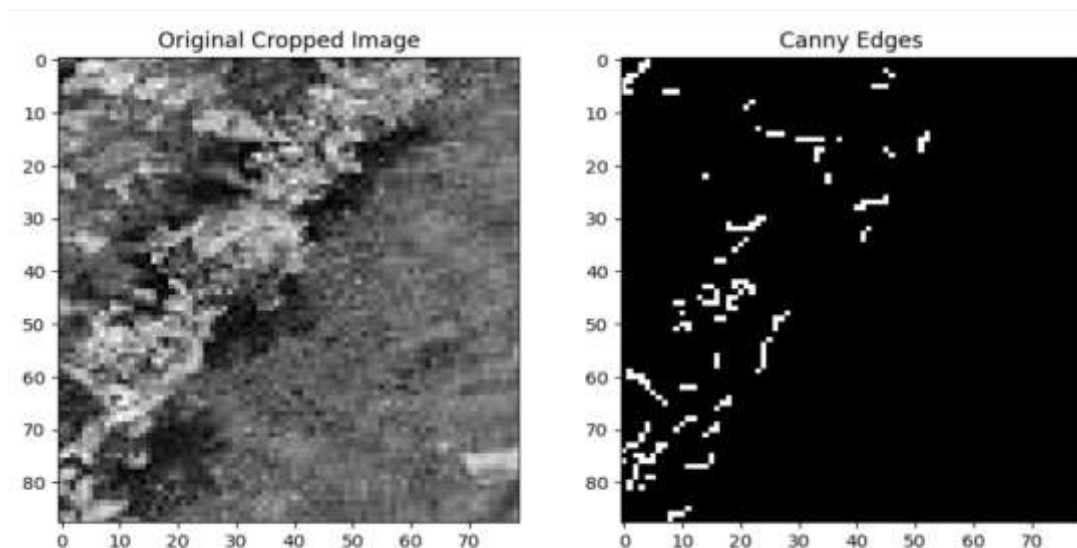
1. Code [Question 01 code](#)

```
import cv2 as cv import numpy as np import matplotlib.pyplot as plt
# Load the image directly from the storage
img = cv.imread('img.jpg', cv.IMREAD_GRAYSCALE)

# Check if the image loaded successfully
if img is None:
    print("Error: Image not found. Check the upload and filename.")
else:
    # Apply Canny Edge Detector
    edges = cv.Canny(img, 550, 690)
    # Extract x and y coordinates
    indices = np.where(edges != 0)
    x = indices[1] y = indices[0]

# Plot original and edge image
fig, ax = plt.subplots(1, 2, figsize=(10, 5)) ax[0].imshow(img, cmap='gray')
ax[0].set_title('Original Image') ax[1].imshow(edges, cmap='gray')
ax[1].set_title('Canny Edges') plt.show()
```

Output



2. Code [Question 02 code](#)

```
import cv2 as cv import numpy as np import matplotlib.pyplot as plt
img = cv.imread('img.jpg', cv.IMREAD_GRAYSCALE)
if img is not None:
    edges = cv.Canny(img, 550, 690)
    indices = np.where(edges != 0)
```

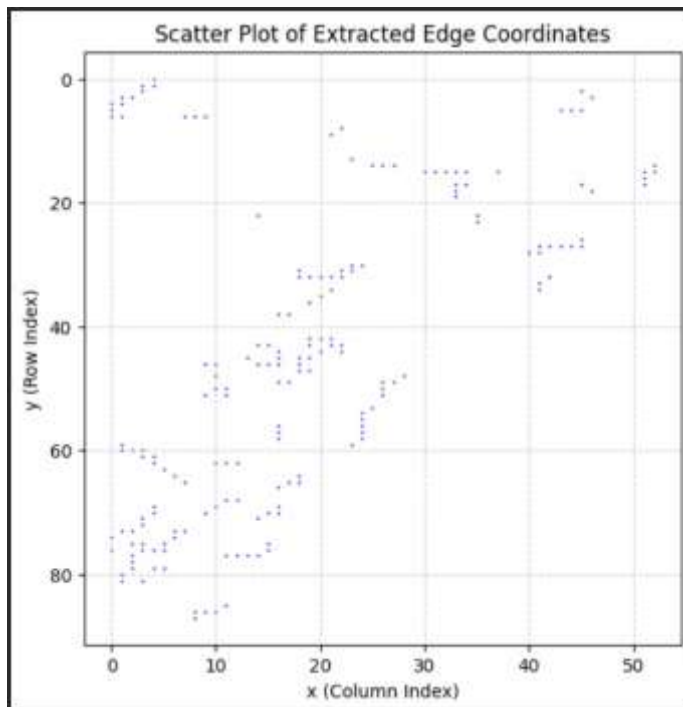
```
x = indices[1] y = indices[0]
```

```
plt.figure(figsize=(6, 6)) plt.scatter(x, y, s=1, c='blue', marker='.')
plt.gca().invert_yaxis() plt.title('Scatter Plot of Extracted Edge Coordinates')
plt.xlabel('x (Column Index)') plt.ylabel('y (Row Index)')
plt.grid(True, linestyle='--', alpha=0.6) plt.show()
```

else:

```
print("Error: Image not found.")
```

Output



3. Code [Question 03 code](#)

```
import cv2 as cv import numpy as np import matplotlib.pyplot as plt
img = cv.imread('img.jpg', cv.IMREAD_GRAYSCALE)
```

if img is not None:

```
edges = cv.Canny(img, 550, 690)
indices = np.where(edges != 0)
x = indices[1] y = indices[0]
```

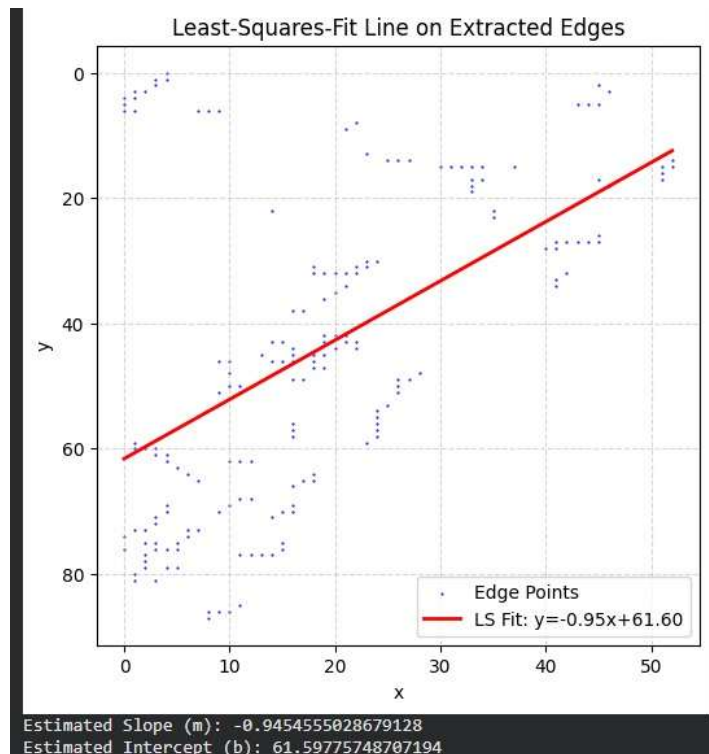
```
m, b = np.polyfit(x, y, 1)
```

```
x_fit = np.array([np.min(x), np.max(x)]) y_fit = m * x_fit + b
```

```
plt.figure(figsize=(6, 6)) plt.scatter(x, y, s=1, c='blue', marker='.', label='Edge Points')
plt.plot(x_fit, y_fit, color='red', linewidth=2, label=f'LS Fit: y={m:.2f}x+{b:.2f}')
```

```
plt.gca().invert_yaxis() plt.title('Least-Squares-Fit Line on Extracted Edges')
plt.xlabel('x') plt.ylabel('y') plt.legend() plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
print(f'Estimated Slope (m): {m}') print(f'Estimated Intercept (b): {b}')
else:
    print("Error: Image not found.")
```

Output



4.

$$\tan(\theta) = m$$

$$\theta = \arctan(m)$$

$$\theta = \arctan(-0.9455)$$

$$\theta = 43.39^\circ$$

5.

This estimation is not correct. Although it gives a mathematical result, it does not accurately represent the true orientation of the crop row for several technical reasons.

The main problem is its sensitivity to outliers. As seen in the scatter plot, many stray edge pixels from weeds, shadows, and soil texture do not belong to the crop row. Because the **Least Squares** method minimizes the squared error, located far from the

main row can strongly influence the fitted line, pulling the fitted line away from the actual row and biasing the slope.

Another problem is that there is an assumption that there is some form of noise, but it is assumed that it is on the Y-axis. The standard equation for a straight line is $y = mx + b$, and it is assumed that there is no error on the X-axis. However, due to errors in image processing, there is a possibility of error on both axes. As a result, there is rotational bias

Another problem is that it is not robust. The inclusion of extra data, such as background and multiple lines, is a major problem. The **Standard Least Squares** method is equal for every point, and as a result, it cannot differentiate between actual and unwanted data.

6. Code [Question 06 code](#)

```
import cv2 as cv import numpy as np import matplotlib.pyplot as plt

img = cv.imread('img.jpg', cv.IMREAD_GRAYSCALE)

if img is not None:

    edges = cv.Canny(img, 550, 690) indices = np.where(edges != 0)

    x = indices[1] y = indices[0]

    x_mean = np.mean(x) y_mean = np.mean(y) x_centered = x - x_mean y_centered = y - y_mean

    data = np.vstack([x_centered, y_centered]).T

    u, s, vh = np.linalg.svd(data)

    a, b = vh[1, :]

    m_tls = -a / b

    c_tls = y_mean - m_tls * x_mean

    plt.figure(figsize=(6, 6)) plt.scatter(x, y, s=1, c='blue', marker='.', label='Edge Points')

    x_plot = np.array([np.min(x), np.max(x)]) y_plot = m_tls * x_plot + c_tls

    plt.plot(x_plot, y_plot, color='green', linewidth=2, label=f'TLS Fit: y={m_tls:.2f}x+{c_tls:.2f}')

    plt.gca().invert_yaxis() plt.xlabel('x') plt.ylabel('y') plt.title('Total Least-Squares-Fit Line')

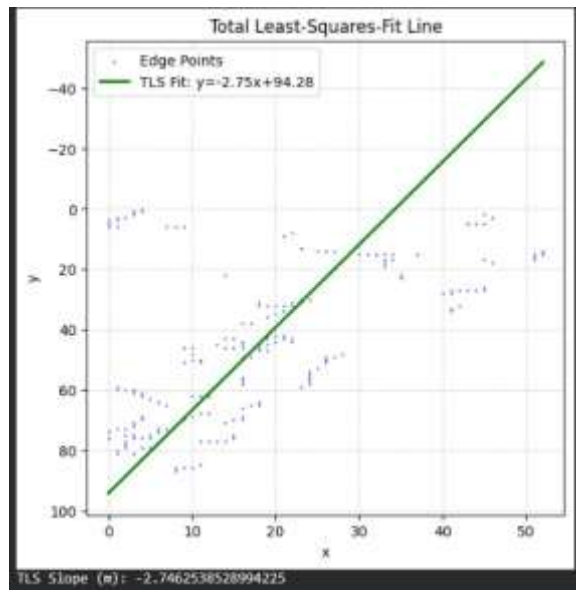
    plt.legend() plt.grid(True, linestyle='--', alpha=0.5) plt.show()

    print(f"TLS Slope (m): {m_tls}")

else:

    print("Error: Image not found.")
```

Output



7.

$$\tan(\theta) = m$$

$$\theta = \arctan(2.7462538528994225)$$

$$\theta = 69.99^\circ$$

8. The estimation is incorrect for accurate determination of the crop row. Although it is a mathematical improvement over **Standard Least Squares**, it is incorrect due to the following reasons.

Firstly, there is still a problem of sensitivity to clutter. Although it is fairer, as it is minimizing the perpendicular distances instead of vertical distances, it is still a global fit. The algorithm utilizes every single one of the extracted edge pixels. The problem is that there is a lot of clutter from weeds and soil patterns. As a result, the line is pulled away from the actual crop row due to the fact that, under **Total Least Squares**., these outliers are considered actual data.

And also, there is a problem of robustness. A robust fitting method should have the ability to differentiate between actual points and outliers. The problem is that there is no such ability to ignore these points in a complex scene, as there could be as many pixels from the actual noise as there could be from the actual crop row.

9.

Random Sample Consensus algorithm

10. Code [Question 10 code](#)

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.linear_model import RANSACRegressor

img = cv.imread('img.jpg', cv.IMREAD_GRAYSCALE)

if img is not None:

    edges = cv.Canny(img, 550, 690)

    indices = np.where(edges != 0)

    x = indices[1] y = indices[0]

    X = x.reshape(-1, 1)


    ransac = RANSACRegressor(residual_threshold=5.0, random_state=42)

    ransac.fit(X, y)

    m_robust = ransac.estimator_.coef_[0] b_robust = ransac.estimator_.intercept_

    x_fit = np.array([np.min(x), np.max(x)]) y_fit = m_robust * x_fit + b_robust

    inlier_mask = ransac.inlier_mask_ outlier_mask = np.logical_not(inlier_mask)

    plt.figure(figsize=(6, 6))plt.scatter(x[outlier_mask], y[outlier_mask], s=1, c='gray', alpha=0.3,
label='Outliers (Clutter)') plt.scatter(x[inlier_mask], y[inlier_mask], s=1, c='blue',
label='Inliers (Crop Row)')
    plt.plot(x_fit,y_fit,color='red',linewidth=3,label=f'RANSAC Fit: y={m_robust:.2f}x+{b_robust:.2f}')
    plt.gca().invert_yaxis() plt.xlabel('x') plt.ylabel('y') plt.title('Robust Line Fitting using RANSAC')
    plt.legend() plt.grid(True, linestyle='--', alpha=0.5) plt.show()

    print(f'RANSAC Estimated Slope (m): {m_robust}')

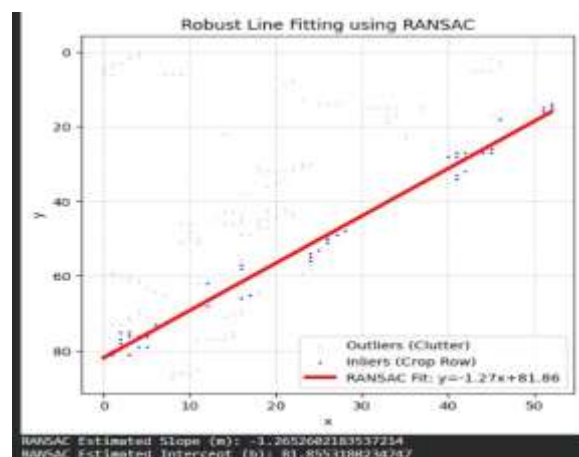
    print(f'RANSAC Estimated Intercept (b): {b_robust}')

else:

    print("Error: Image not found.")

```

Output



11.

$$\theta = \arctan(m)$$

$$\theta = \arctan(1.2652602183537214)$$

$$\theta = 51.67^\circ$$

12. The performance of the **RANSAC** algorithm is better because it is a robust estimator, and it has been designed to deal with a set of data that has a high percentage of outliers, such as weeds and soil in the image.

The major advantage of inlier and outlier discrimination is that it is different from the **Least Squares** and **Total Least Squares** methods, as they are global estimators and try to fit all the points, whereas the **RANSAC** algorithm works on the assumption that there is a set of data with a consensus and some noise, and it tries to find a line that can fit the maximum number of points, ignoring the stray points and aligning the line according to its orientation.

Another reason is that it eliminates the squared error penalty. In the **Least Squares** and **Total Least Squares**, the error is squared. This means that a single pixel of a weed that is far from a row has a huge effect on the result. In **RANSAC**, a threshold on distances is used. This means that if a point is too far away, it is ignored. This prevents a line being fit to an outlier, no matter how far it is from a row.

And also, **RANSAC** effectively handles the structured noise. Scenes containing complex patterns like clumps of weeds or shadows that confuse standard averaging methods. By focusing only on the subset of points that agree on a single linear model, **RANSAC** provides a much more accurate and stable estimation of the crop field angle.